

FoodBridge — React Frontend Mentorship

Day 3 Notes — Register Fixes, Axios Setup, CORS, and Real Login

1. Backend Schema Alignment

Received actual backend user table columns and matched frontend state names exactly:

```
id (auto, ignore), full_name, email, password, phone_number,  
address, organization_name (optional), role
```

Register form useState fields created to mirror this (camelCase, matching backend JSON keys):

```
const [fullName, setFullName] = useState("");  
const [email, setEmail] = useState("");  
const [password, setPassword] = useState("");  
const [phoneNumber, setPhoneNumber] = useState("");  
const [address, setAddress] = useState("");  
const [organizationName, setOrganizationName] = useState("");  
const [role, setRole] = useState("");
```

2. Register Form — Controlled Inputs & Conditional Field

Wired value + onChange for every field, using the controlled-component pattern from Day 2. Address field uses a textarea instead of input since it's multi-line text:

```
<textarea  
  value={address}  
  onChange={(e) => setAddress(e.target.value)}  
  rows={3}  
</>
```

Organization Name is only rendered when role is NGO, using conditional rendering:

```
{role === "NGO" && (  
  <div>...Organization input...</div>  
)}
```

ADMIN excluded from the public role dropdown — admins are created directly, not via self-registration.

Common mistake caught: copy-pasting input blocks without renaming state variables (e.g. Password field accidentally used phoneNumber/setPhoneNumber). Lesson: always double check variable names after copy-pasting similar JSX blocks.

3. Concept Recap: value={state} (Controlled Inputs)

Whiteboard/mirror analogy used: state (e.g. fullName) is the whiteboard — the real source of truth. The input box is a mirror that only shows what's on the whiteboard, via value={fullName}. onChange writes the user's typing back onto the whiteboard. Without value={state}, the input keeps its own separate memory (uncontrolled) and won't clear or update if state is changed programmatically (e.g. after form reset).

4. Register Submit Handler

```
const handleRegister = (e) => {  
  e.preventDefault()  
  console.log(`Full Name : ${fullName}`)  
  console.log(`Email : ${email}`)  
  console.log(`Password : ${password}`)  
  console.log(`Phone Number : ${phoneNumber}`)
```

```

    console.log(`Address : ${address}`)
    console.log(`Organization Name : ${organizationName}`)
    console.log(`Role : ${role}`)
  }
}

```

e.preventDefault() recap: without it, the browser's default form behavior reloads the entire page, wiping all React state instantly. preventDefault() stops that so React/JS (and later, the Axios API call) handles submission instead.

5. Introduction to Axios

Axios is a library for sending HTTP requests from React to the Spring Boot backend and getting a reply back — same job as fetch, done with less manual work.

Basic shape of a request:

```

axios.post("http://localhost:8080/users/login", {
  email: "test@test.com",
  password: "1234"
})

```

Handling the response (Promise-based):

```

axios.post(url, data)
  .then((response) => {
    console.log(response.data)    // success
  })
  .catch((error) => {
    console.log(error)           // failure
  })

```

- .then() -> runs when the request succeeds; response.data is the actual reply content (auto-parsed JSON, unlike fetch)
- .catch() -> runs when the request fails (network error, validation error, server error, etc.)

6. Centralized Axios Instance

Problem: repeating the full backend URL in every API call is error-prone and hard to maintain (e.g. if the URL changes later, or when JWT headers need to be attached to every request). Solution: create one reusable, pre-configured Axios instance.

Created src/api/axiosInstance.js:

```

import axios from "axios"

const axiosInstance = axios.create({
  baseURL: "http://localhost:8080",
})

export default axiosInstance

```

Backend has no /api prefix — endpoints are directly like /users, /users/login, /donations, per the provided API documentation. Usage elsewhere becomes short, e.g. axiosInstance.post('/users/login', data) instead of writing the full URL every time.

7. Backend API Reference (Key Endpoints)

POST	/users	Register (public)
POST	/users/login	Login (public) -> returns JWT

```

POST    /donations          Create donation (DONOR, JWT required)
GET     /donations/available List available donations (NGO, JWT required)
PUT     /donations/{id}/accept Accept donation (NGO, JWT required)
GET     /donations/accepted Accepted donations (VOLUNTEER, JWT required)
PUT     /donations/{id}/pickup Mark picked up (VOLUNTEER, JWT required)
PUT     /donations/{id}/deliver Mark delivered (VOLUNTEER, JWT required)
GET     /users/admin/test   Admin test (ADMIN, JWT required)

```

Common headers for protected routes: Authorization: Bearer <JWT> and Content-Type: application/json.

8. Connecting Login to the Backend

```

import axiosInstance from "../../api/axiosInstance"

const handleSubmit = (e) => {
  e.preventDefault()
  axiosInstance.post("/users/login", { email, password })
    .then((response) => {
      console.log(response.data)
    })
    .catch((error) => {
      console.log(error.response.data)
    })
}

```

Mistake caught: initially wrote handleSubmit without the (e) parameter and without e.preventDefault(), which would cause a full page reload on submit, wiping state before the response could be logged.

Also learned: both frontend (npm run dev, port 5173) and backend (Spring Boot, port 8080) must be running at the same time — they are two separate programs communicating over HTTP.

9. Debugging CORS Error

Error seen: 'blocked by CORS policy: No Access-Control-Allow-Origin header'. Browsers block a webpage on one origin (localhost:5173) from calling a server on a different origin (localhost:8080) unless the server explicitly allows it. This is a backend-side fix, not a frontend one.

Fix applied in Spring Security's SecurityConfig.java:

```

.cors(cors -> cors.configurationSource(corsConfigurationSource()))

@Bean
public CorsConfigurationSource corsConfigurationSource() {
  CorsConfiguration configuration = new CorsConfiguration();
  configuration.setAllowedOrigins(List.of("http://localhost:5173"));
  configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
  configuration.setAllowedHeaders(List.of("*"));
  configuration.setAllowCredentials(true);

  UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
  source.registerCorsConfiguration("/**", configuration);
  return source;
}

```

Note: Spring Security ignores CORS config unless .cors(...) is explicitly enabled inside the SecurityFilterChain, even if a CorsConfigurationSource bean exists elsewhere.

10. Debugging 400 Bad Request

After CORS was fixed, login returned 400 Bad Request. To see the real reason instead of a generic Axios error, updated the catch block:

```
.catch((error) => {  
  console.log(error.response.data)  
})
```

Root cause turned out to be incorrect test credentials, not a code bug — confirmed once logging in with a real registered user returned a valid JWT token successfully.

11. Milestone Reached

Full end-to-end flow now works: React form -> Axios request -> Spring Boot validation -> CORS allows it through -> JWT token returned and logged in browser console.

12. Git Commit (End of Day 3)

```
git add .  
git commit -m "feat: complete Register form, integrate Axios, connect Login to backend API  
  
- Added conditional Organization Name field (shown only when role === NGO)  
- Implemented useState across all Register fields matching backend schema  
- Added handleRegister with e.preventDefault() and field logging  
- Installed and configured Axios with centralized axiosInstance (baseUrl: http://localhost:8080)  
- Connected Login form to POST /users/login endpoint  
- Fixed CORS by adding CorsConfigurationSource bean in Spring Security config  
- Verified successful login flow returning real JWT token from backend"
```

13. Next Steps (Day 4)

- Create AuthContext to store JWT + logged-in user + role globally
- Store JWT securely (localStorage) after successful login
- Redirect user to correct dashboard based on role after login
- Attach JWT automatically to protected requests via Axios interceptor
- Build ProtectedRoute component for role-based route access
- Connect Register form to POST /users endpoint (same pattern as Login)